Dipartimento di Informatica
Laboratorio di Architettura degli Elaboratori 1
(LAE1)



UNIVERSITÀ DEGLI STUDI
DI MILANO

Introduzione a VHDL

Docenti del corso

Matteo Rè matteo.re@unimi.it
Massimo Walter Rivolta massimo.rivolta@unimi.it
Gabriella Trucco gabriella.trucco@unimi.it

Ambiente software esercitazioni in laboratorio

L'ambiente di esercitazione ha le seguenti caratteristiche:

- Avviare la macchina col sistema operativo LINUX (portare il mouse in alto e centro schermo e cliccare sulla «X», all'avvio selezionare linux)
- E' un ambiente di tipo **UNIX** (ad essere precisi Fedora Linux 12).
- Una volta avviato vi chiede di inserire delle credenziali di accesso.
  - Nome utente: **email di Ateneo**
  - Password: **password personale**
- In esso si lavora **usando la riga di comando**.
- Nonostante il fatto che molti passi del lavoro in VHDL (analisi/elaborazione) vengano effettuati da riga di comando, avrete a disposizione un ambiente grafico (finestre/cartelle/blocco note*)

LA PRIMA COSA DA FARE DOPO ESSERE ENTRATI NELL'AMBIENTE DI ESERCITAZIONE E' APRIRE IL PROMPT DEI COMANDI!

Cosa è VHDL?

- VHSIC (Very High Speed Integrated Circuit) Hardware Description Language è un linguaggio per descrivere hardware.

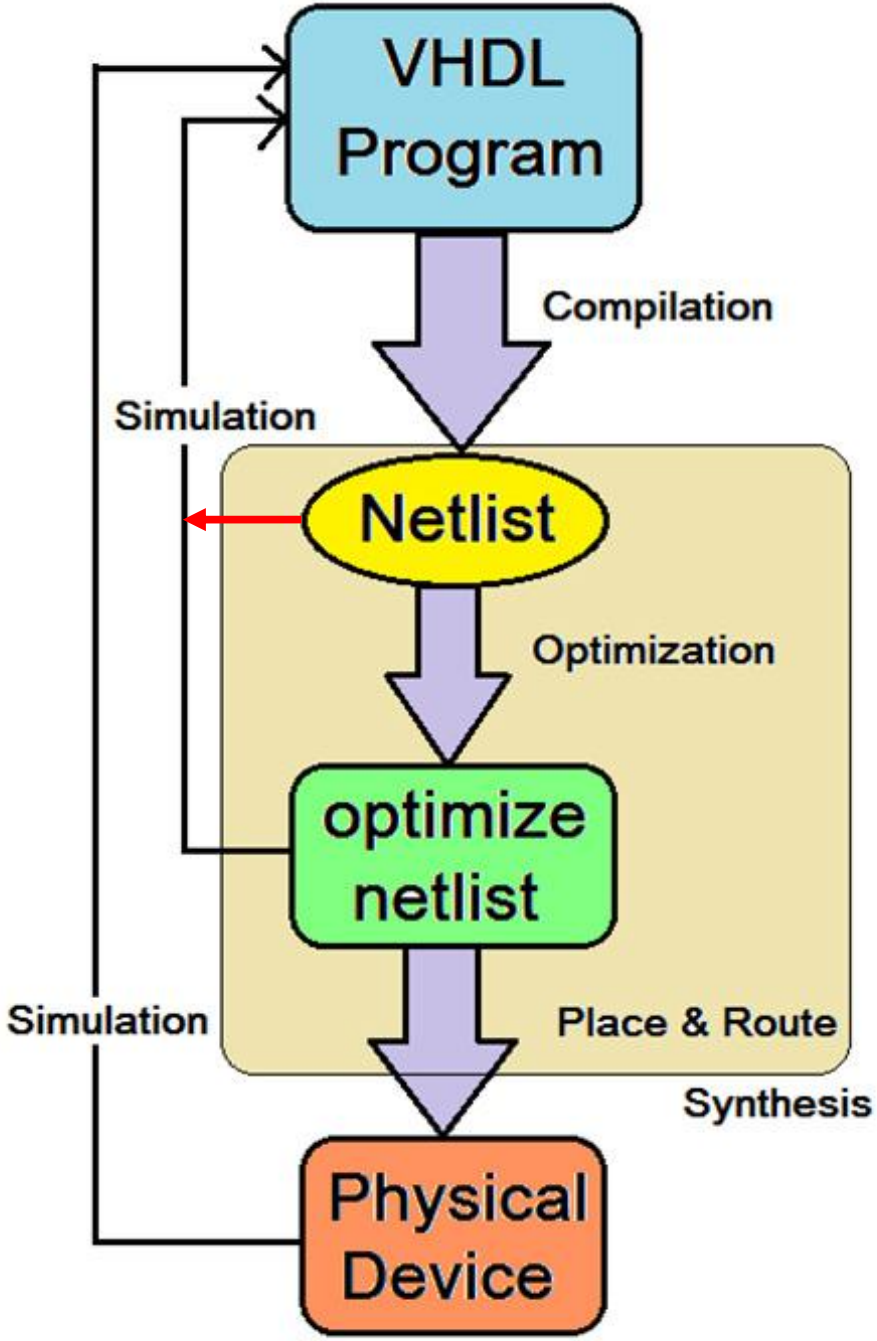
- VHDL non è un linguaggio eseguibile: non specifica quali operazioni un esecutore deve eseguire per ricavare il risultato di una elaborazione, ma descrive gli elementi che costituiscono il circuito digitale in grado di effettuare l'elaborazione richiesta.

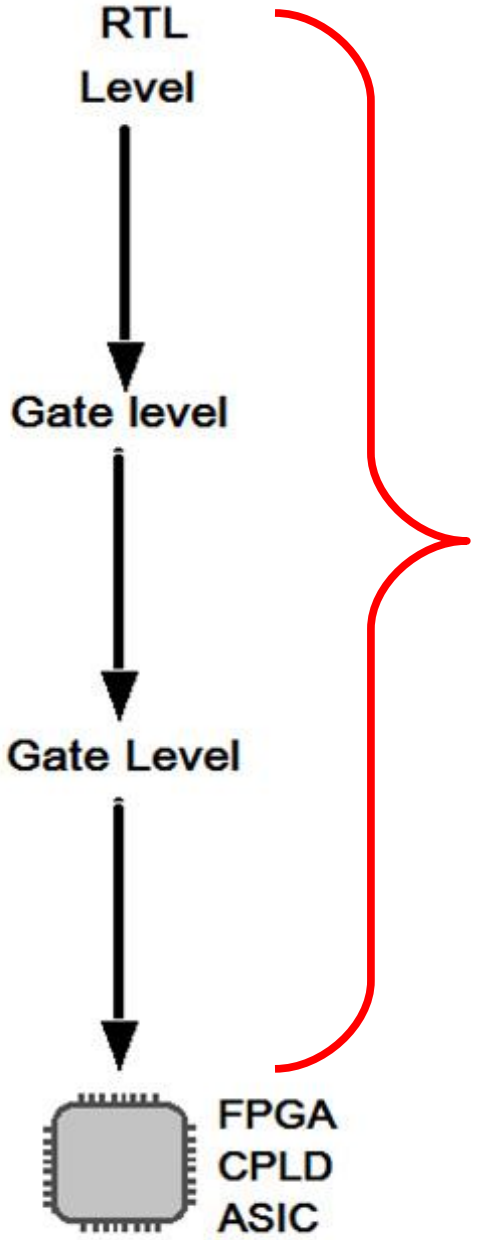
- Una specifica VHDL può essere simulata con opportuni strumenti. Simulare una specifica VHDL = simulare il comportamento del circuito.

- VHDL permette di descrivere il circuito a diversi livelli di astrazione.

VHDL ciclo di lavoro:

Noi vedremo solo descrizione e simulazione. NO sintesi !!!





Flusso di progettazione digitale

1. **Specifica e descrizione funzionale**
   - Si definiscono i requisiti del circuito (cosa deve fare, a che velocità, numero ingressi/uscite, …).
   - Il progettista scrive il codice VHDL (o Verilog) per descrivere il circuito, lavorando principalmente a livello behavioural e RTL (Register-Transfer Level). Questo codice definisce il **comportamento logico**.

2. **Simulazione e verifica funzionale**
   - Verificare che il codice VHDL si comporti logicamente come desiderato, senza considerare i dettagli fisici del chip (come i ritardi).
   - Si crea un modulo aggiuntivo, chiamato **Testbench** (scritto in VHDL), che simula gli ingressi al circuito. Un simulatore software esegue il codice per confermare che l'output sia corretto.

3. **Sintesi (traduzione e ottimizzazione)**
   - Convertire la descrizione di alto livello in una rappresentazione fisica.
   - Lo strumento di sintesi legge il codice VHDL (purché sia sintetizzabile), lo ottimizza (per velocità, area e potenza) e lo traduce in una Netlist al Gate-Level (Netlist: lista di porte logiche interconnesse, specifiche per la tecnologia del chip).

Flusso di progettazione digitale

4. Implementazione fisica (Place & Route)

- Mappare la Netlist ottimizzata sulla geometria fisica del dispositivo target (FPGA o ASIC).
  - Piazzamento (Place): le porte logiche della Netlist vengono assegnate a blocchi logici specifici.
  - Instradamento (Route): vengono disegnate le connessioni elettriche reali tra i blocchi.
- Ruolo del VHDL: nessuno, o indiretto. Questa fase è gestita esclusivamente dagli strumenti software del produttore del chip e si basa sulla netlist generata dalla sintesi.

5. Simulazione post-layout e caricamento

- Verifica che il circuito funzioni correttamente *dopo* l'implementazione fisica, tenendo conto dei ritardi effettivi causati dall'instradamento (i "fili" sulla scheda).
- Vengono estratti i dati di temporizzazione fisica e si esegue una simulazione finale. Se fallisce, si deve tornare al codice VHDL (Step 1) e modificarlo per risolvere i problemi di temporizzazione.
- Risultato finale: un file Bitstream (per FPGA) o un file GDSII (per ASIC) pronto per la fabbricazione.

VHDL: livelli di astrazione

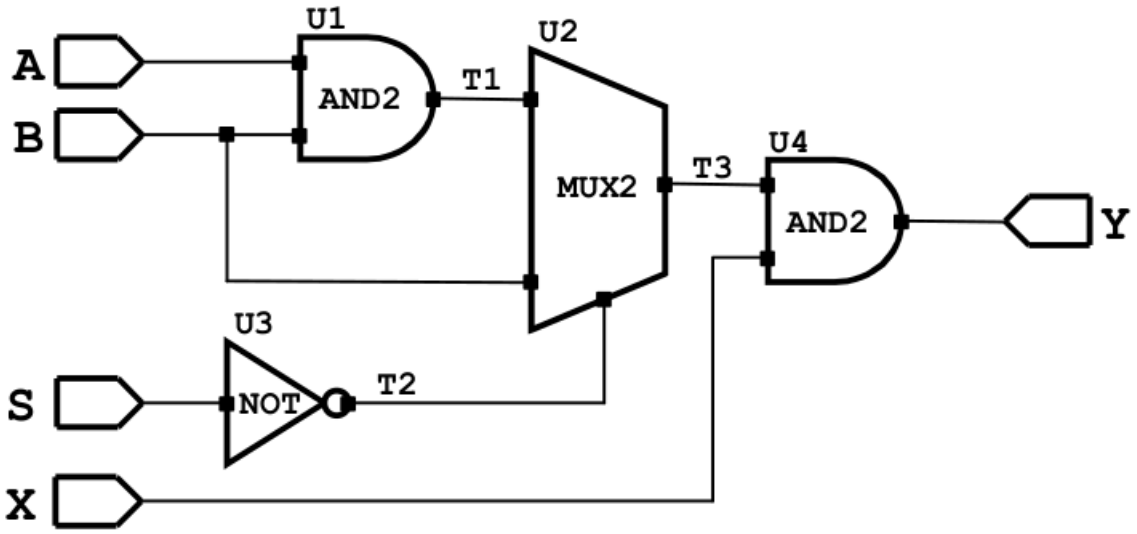
VHDL è in grado di **descrivere l'hardware in modo diversi**.

- Può utilizzare uno stile di descrizione estremamente vicino all'hardware (in questo caso parliamo di un **basso** livello di astrazione)
- o un livello di descrizione **estremamente astratto**, più comodo poiché più "vicino" al modo di pensare degli esseri umani (immaginatelo come più vicino all'algebra che alle porte logiche) ma non direttamente traducibile in circuito.

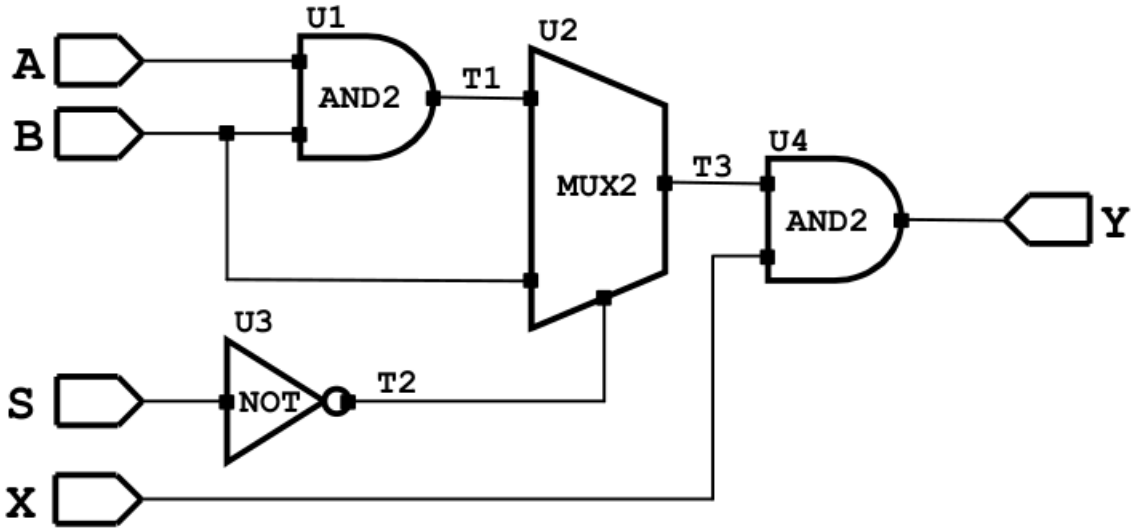
Iniziamo a vedere i vari livelli di astrazione forniti da VHDL…

VHDL: livello di astrazione gate-level

- Livello di astrazione più basso (più vicino all'hardware). Il circuito è **descritto** come un grafo in cui i nodi sono **elementi logici** (semplici o complessi: porte, multiplexer, sottocircuiti…) mentre gli archi sono le **connessioni** tra elementi.
- Le operazioni possibili sono solo quelle ammesse dagli elementi di base (AND, OR, NOT, flip-flop, …)



VHDL: livello di astrazione gate-level



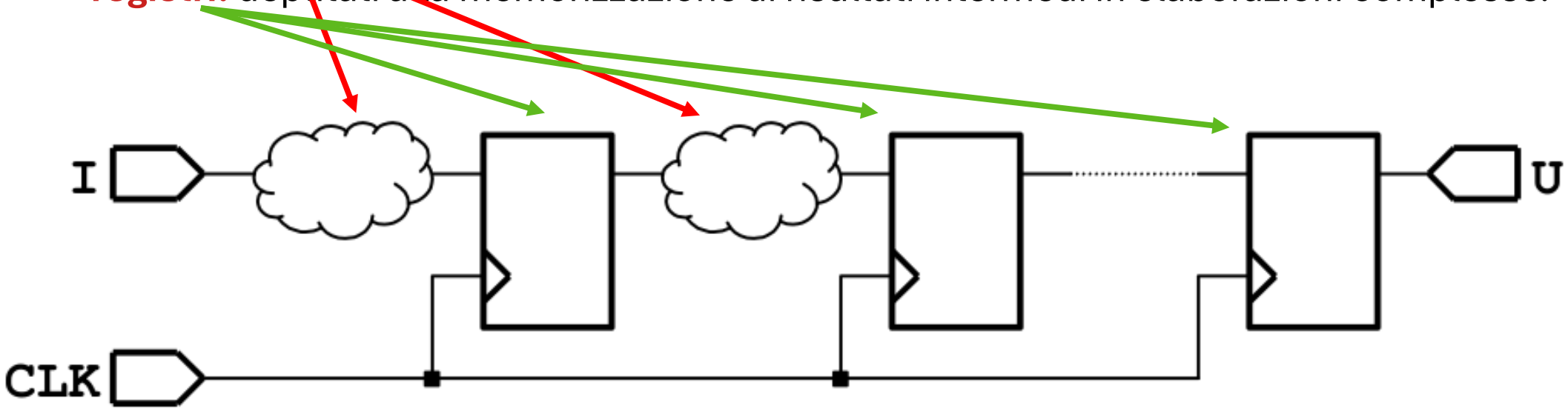
INFORMAZIONI CONTENUTE:

- Nomi degli ingressi/uscite primari del dispositivo (**A,B,S,X,Y**)
- Tipo degli elementi logici (**AND2, NOT, MUX2**)
- Nomi delle istanze degli elementi logici (**U1,U2,U3,U4**)
- Nomi dei **segnali interni** (**T1,T2,T3**)
- **Connessioni** tra segnali e porte dei componenti

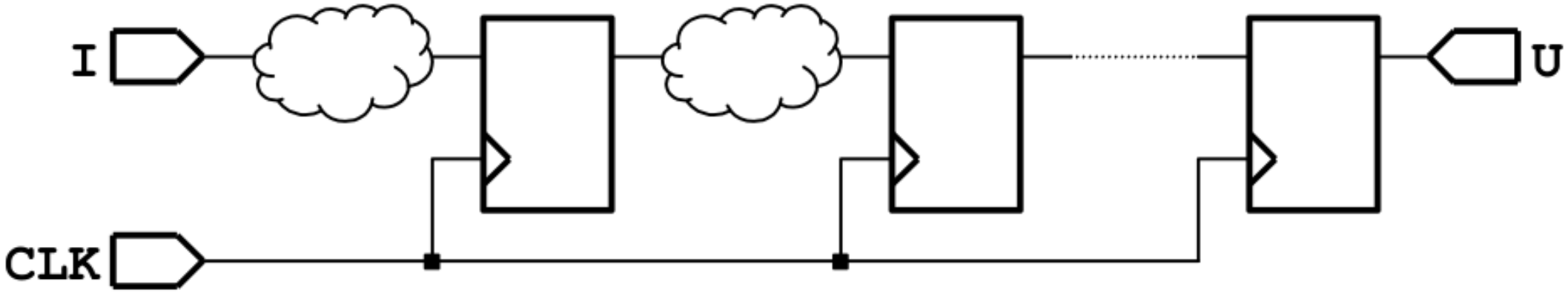
VHDL: livello register-transfer RTL

- Livello di astrazione intermedio detto anche **data-flow**.

- Il circuito è visto come l'insieme di due tipi di elementi:
  - **reti combinatorie:** esprimono esplicitamente le trasformazioni dei dati;
  - **registri:** deputati alla memorizzazione di risultati intermedi in elaborazioni complesse.



VHDL: livello di astrazione RTL



INFORMAZIONI CONTENUTE:

- Generici insiemi di ingressi e di uscite (**I,U**)
- Posizione reti combinatorie di complessità arbitraria (**nuvolette**)
- Posizione generici elementi di memoria (**register**)
- Esprime trasformazioni di segnali complesse (es. somme) e punti di memorizzazione
- Ogni operazione è assegnata ad un **preciso** stadio di elaborazione ovvero ad uno **specifico ciclo di clock**. Assegnamento di una operazione ad un ciclo di clock → **SCHEDULING**

VHDL: livello algoritmico o behavioral

- Livello di astrazione più alto (più distante dall'hardware) permesso da VHDL.
- Viene descritta la funzionalità di un circuito mediante **uno o più algoritmi**.
- Si usa per modellare il comportamento logico del circuito nel tempo, in risposta a determinati segnali di ingresso.
- **Né la struttura né le trasformazioni dei dati sono esplicite**. Non è esplicito come operazioni/elaborazioni siano **assegnate ai cicli di clock**. Non è possibile mostrare uno schema di struttura come per i livelli gate-level e RTL.
- Sarà un apposito **strumento (automatizzato) di sintesi** a ricavare la struttura circuitale sulla base di specifici **vincoli** imposti dal progettista (ad esempio frequenza di clock minima).

VHDL: regole di sintassi generali

- VHDL è case insensitive (non distingue minusCole/maiuscole)
- Tutti i nomi **devono** iniziare con un carattere alfabetico [a-z,A-Z]
- Caratteri **permessi**: [a-z,A-Z] , cifre (0-9) e _
- **NON** usare segni di punteggiatura (es. ! , ? , . , ; , &, +, -)
- **NON** usare due o più _ in un nome (es. Sel__a non è valido)
- **Tutti** i nomi/etichette in una entità o architettura **DEVONO** essere unici.
- Non ci sono vincoli di formato. **Spazio** e **a capo** sono equivalenti **ma ogni riga finisce con ;** (a parte rare eccezioni)

VHDL: regole generali

- Le seguenti forme sono equivalenti (no vincoli formato):

```
if (a=b) then
```

or

```
if  (a=b)          then
```

or

```
if (a =
b) then
```

Tipi di dato fondamentali

bit e *bit_vector*

- Possono assumere solo i valori '0' e '1'.
- *bit_vector*: un array di bit.
- Sono tipi *predefiniti* in VHDL, ma sono meno robusti per la sintesi e la simulazione.
- **Sconsigliati per progetti reali** in favore di *std_logic*.

Tipi di dato fondamentali

std_logic

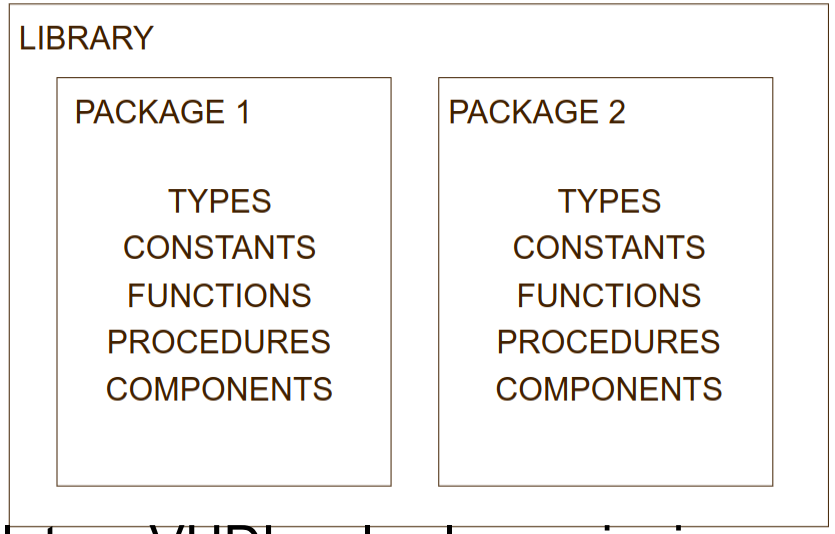
- Il tipo di dato più importante per la descrizione hardware.
- Definito nel pacchetto IEEE.STD_LOGIC_1164.
- Può assumere 9 valori, non solo '0' e '1'. Questo è fondamentale per una simulazione realistica:
  - '0' : forza logica bassa
  - '1' : forza logica alta
  - 'Z' : alta impedenza (segnale flottante)
  - 'X' : sconosciuto / indeterminato (conflitto)
  - 'U' : non inizializzato
  - 'W' : debole sconosciuto
  - 'L' : debole bassa
  - 'H' : debole alta
  - '-' : don't care (non importa)
- Questi stati permettono di modellare situazioni reali come bus in tristate, conflitti di segnale, ecc.

Struttura generale di un file VHDL:

Ogni file VHDL di solito segue una struttura ben definita.

- **Librerie (library e use)**
- **Dichiarazione di entity**
- **Dichiarazione di architecture (della entity)**

Struttura generale librerie VHDL



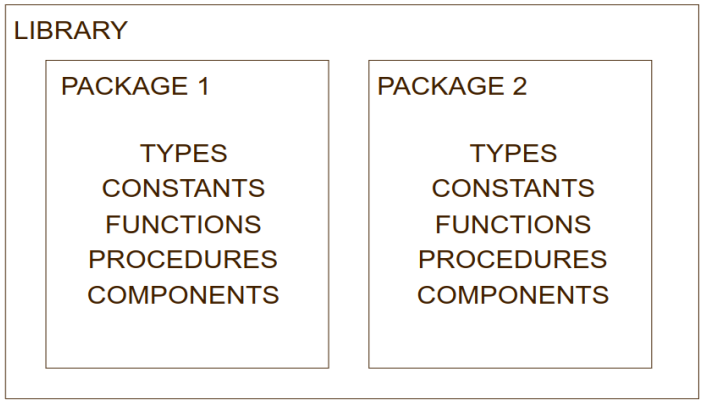
Libreria: è l'unità più grande di archiviazione in VHDL.

- È una directory o una cartella logica sul disco rigido dove il compilatore VHDL salva le versioni compilate dei moduli di progetto.
- Fornisce un meccanismo per organizzare il codice compilato e renderlo disponibile ad altri progetti.
- La libreria di default e più importante in VHDL è la **IEEE** (che contiene il package std_logic_1164).
- Per usare il contenuto di una libreria in un file VHDL, è necessario usare la clausola:
  LIBRARY nome_libreria;

Package: contenitore principale di elementi di design che si desidera condividere.

- È contenuto *all'interno* di una Libreria.
- Funziona come un "header file" nei linguaggi di programmazione software (come C/C++), dichiarando tutti gli elementi riutilizzabili.
- Per usare il contenuto di un package in un'entità o architettura VHDL, si usa la clausola:
  USE nome_libreria.nome_package.all;

Struttura generale librerie VHDL



Ogni package può contenere una combinazione dei seguenti elementi, che rappresentano gli elementi base per il riuso del codice:
TYPES (Tipi): nuovi tipi di dati definiti dal progettista
CONSTANTS (Costanti): valori fissi che non cambiano durante l'esecuzione
FUNCTIONS (Funzioni): sottoprogrammi che calcolano un valore di output e lo restituiscono
PROCEDURES (Procedure): sottoprogrammi che possono eseguire una serie di operazioni complesse e modificare i parametri passati.
COMPONENTS (Componenti): le dichiarazioni delle interfacce (porte) di altre entità VHDL. Questo è fondamentale per instanziare blocchi già progettati

La struttura **Libreria** → **Package** → **Elementi** permette:

- **Modularità:** dividere un progetto complesso in parti gestibili.
- **Riutilizzo:** definire un elemento (come un tipo o una funzione) una sola volta e usarlo in più punti del progetto.
- **Gerarchia:** organizzare i file sorgente in modo ordinato per progetti su larga scala.

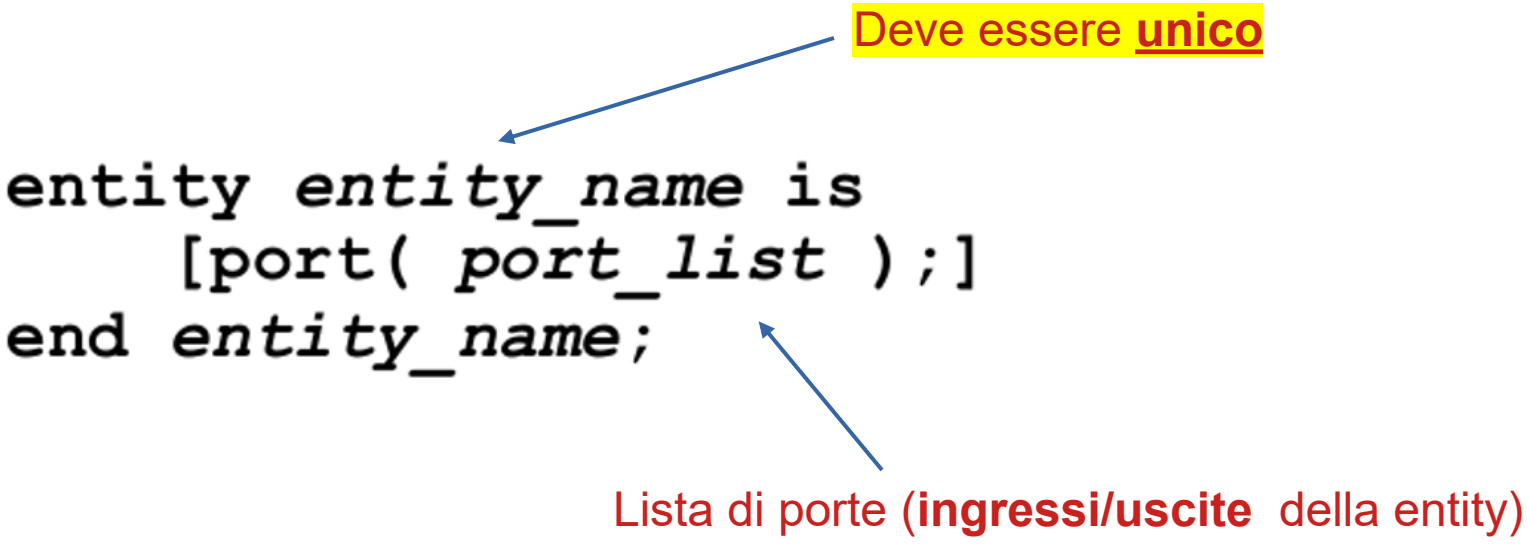
Struttura generale di un file VHDL: librerie

- **Librerie (parole riservate library e use):**
  - Servono per importare intere librerie o pacchetti di funzioni e tipi predefiniti.
  - Le più comuni:
    - IEEE: contiene standard logici e operazioni aritmetiche.
    - STD: contiene tipi standard VHDL.
  - Esempio:

```
library ieee;
use ieee.std_logic_1164.all; -- Pacchetto per std_logic e std_logic_vector
```

Struttura generale di un file VHDL: entity

- **Entity (entity)**
  - È l'interfaccia **esterna** del nostro modulo/componente hardware
  - Definisce i "pin" del nostro circuito: ingressi (input) e uscite (output)
  - **Sintassi**:



Struttura generale di un file VHDL: entity

port(port_list);

- Descrive i segnali che attraversano gli **ingressi/uscite** della entity
- Ogni dichiarazione di porta segue questa sintassi

```
port_name[,port_name,...]: {in|out|inout} port_type;
```

- Uno o più **port_name** separati da , : i nomi della/delle porte
- Direzione della porta: **in|out|inout**
- **port_type** : tipo (di dato) della porta/delle porte dichiarate

Struttura generale di un file VHDL: entity

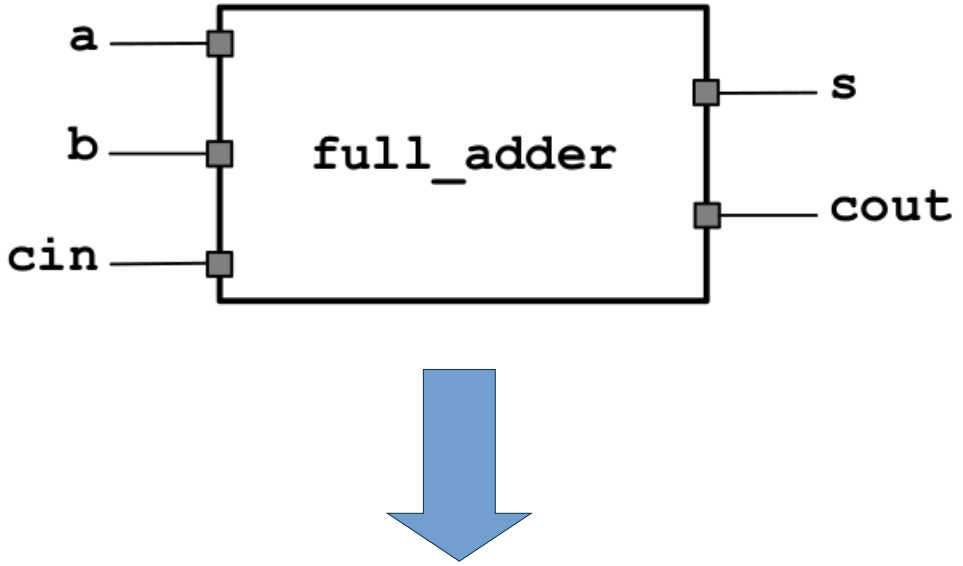
port(port_list);

- Descrive i segnali che attraversano gli **ingressi/uscite** della entity
- Ogni dichiarazione di porta segue questa sintassi

```
port_name[,port_name,...]: {in|out|inout} port_type;
```

- Uno o più **port_name** separati da , : i nomi della/delle porte
- Direzione della porta: **in|out|inout**
- **port_type** : tipo (di dato) della porta/delle porte dichiarate
- **WARNING:** L'ultima dichiarazione di porta (o di gruppo di porte) tra le parentesi tonde di **port( )** **NON deve essere seguita dal ;**

Struttura generale di un file VHDL: entity

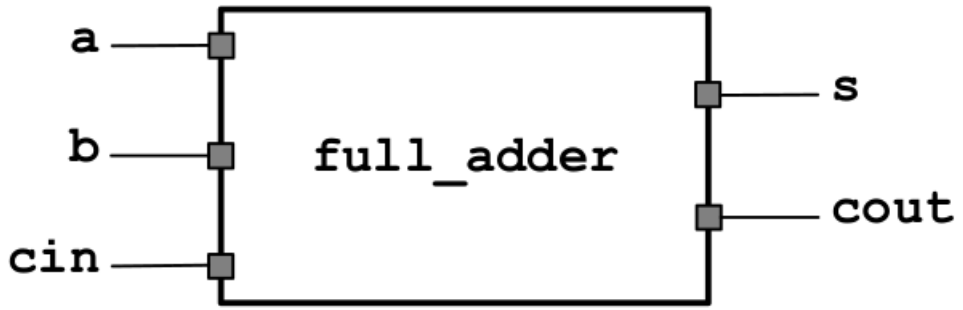



```
entity full_adder is
    port( a, b, cin: in  bit;
          s, cout:   out bit );
end full_adder;
```

Struttura generale di un file VHDL: entity

```
entity full_adder is
    -- Ports
    port(
       -- Inputs
       a:      in  bit;     -- First operand
       b:      in  bit;     -- Second operand
       cin:    in  bit;     -- Carry-in
       -- Outputs
       s:      out bit;     -- Sum
       cout:   out bit      -- Carry-out
    );

end full_adder;
```



COMMENTI iniziati da --

Struttura generale di un file VHDL: architecture

- Entity: descrive **l'interfaccia** del modulo
- Architecture: descrive la **funzione** del modulo e **come questa è realizzata**.
  - Può descrivere reti combinatorie o sequenziali.

- Sintassi:

```
architecture architecture_name of entity_name is
    [declarations]
begin
    [implementation]
end architecture_name;
```

Struttura generale di un file VHDL: architecture

```
architecture architecture_name of entity_name is
    [declarations]
begin
    [implementation]
end architecture_name;
```



Una qualsiasi architecture è associata **ad una ed una sola** entity!

Gli entity_name **DEVONO** essere unici e l'entity_name **fa parte** della dichiarazione di architecture!

NON E' VERO IL CONTRARIO: è possibile associare diverse architecture ad una stessa entity e **scegliere quale usare** prima di procedere con la simulazione.

Struttura generale di un file VHDL: architecture

```
architecture architecture_name of entity_name is
    [declarations]
begin
    [implementation]
end architecture_name;
```

[declarations]: opzionale e costituita da un elenco di dichiarazioni. Le dichiarazioni possono essere di **4 tipi**:

1) Dichiarazioni di **costanti**: nome, tipo, valore di costanti simboliche

2) Dichiarazione di **segnali**: nomi e tipi dei senali che saranno utilizzati

3) Dichiarazione di **tipi**: nomi e definizione di tipi definiti dall'utente

4) Dichiarazione di **componenti**: nomi di interfacce/moduli che saranno usati

Struttura generale di un file VHDL: architecture

```
architecture architecture_name of entity_name is
    [declarations]
begin
    [implementation]
end architecture_name;
```

[implementation] : contiene l'implementazione della funzionalità che l'entity deve svolgere.

La funzionalità di un intero circuito si trova quindi distribuita tra le architecture delle diverse entity che compongono il circuito.

Struttura generale di un file VHDL: entity e architecture

```
entity full_adder is
    generic( delay: real );
    port( a, b, cin: in  bit;
          s, cout:   out bit );
end full_adder;

architecture first of full_adder is
begin
    s <= a xor b xor cin;
    cout <= (a and b) or (b and cin) or (a and cin);
end first;
```

Struttura generale di un file VHDL: entity e architecture

```
entity full_adder is
    generic( delay: real );
    port( a, b, cin: in  bit;
          s, cout:   out bit );
end full_adder;

architecture first of full_adder is
begin
        s <= a xor b xor cin;
        cout <= (a and b) or (b and cin) or (a and cin);
end first;
```

Esempio VHDL: nand_gate



a	b	z
0	0	1
0	1	1
1	0	1
1	1	0

Esempio VHDL: nand_gate

3 sezioni nel file VHDL. Il nome del file deve essere lo stesso della entity.

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY nand_gate IS
     PORT(
          a    : IN STD_LOGIC;
          b    : IN STD_LOGIC;
          z    : OUT STD_LOGIC);
END nand_gate;

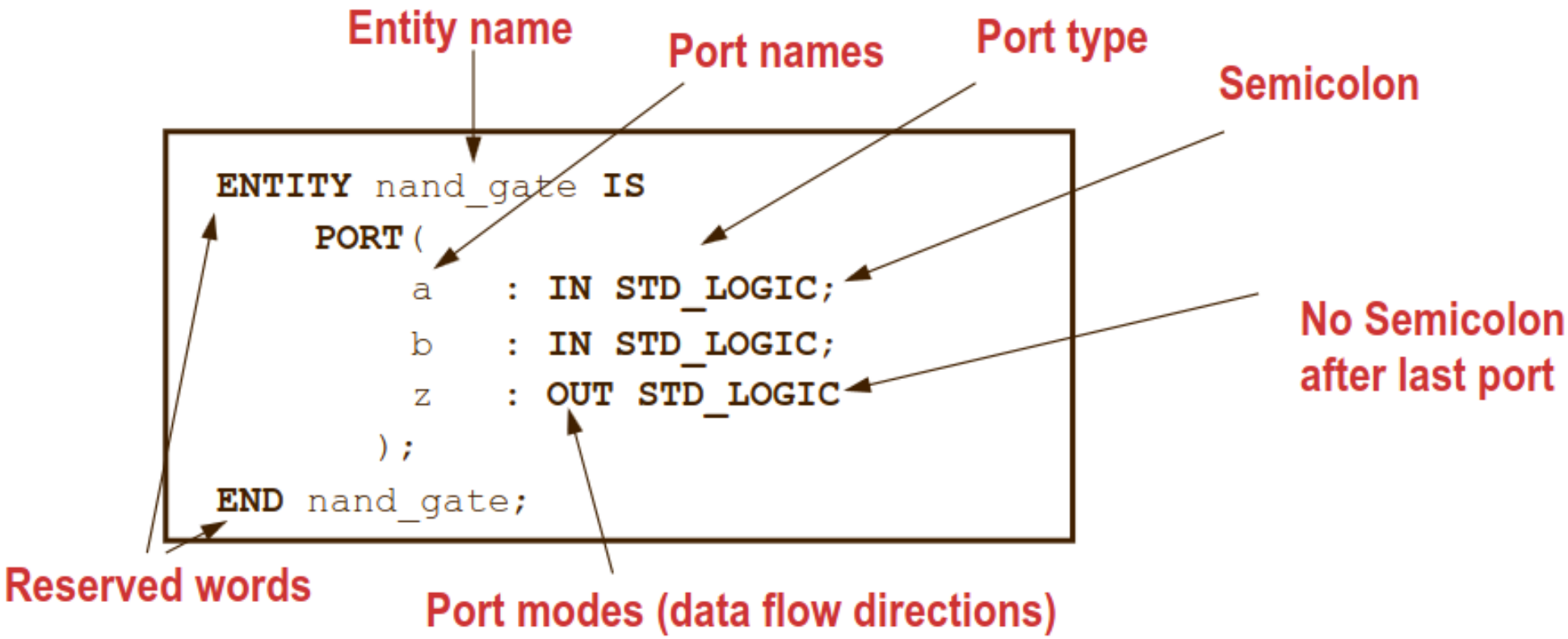
ARCHITECTURE model OF nand_gate IS
BEGIN
     z <= a NAND b;
END model;
```

LIBRARY DECLARATION

ENTITY DECLARATION

ARCHITECTURE BODY

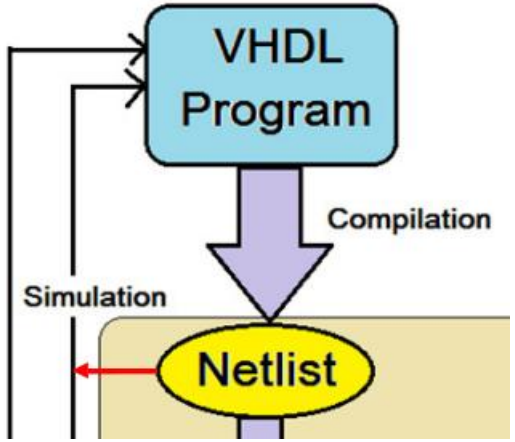
Esempio VHDL: nand_gate



Simulazione: primi concetti

- Cos'è la simulazione: eseguire il design VHDL in un ambiente software per verificarne il comportamento prima della realizzazione fisica.
- Perché è importante?
  - Trovare errori di design in fase iniziale.
  - Testare diverse condizioni di ingresso.
  - Risparmiare tempo e costi rispetto alla prototipazione fisica.

Processo di simulazione

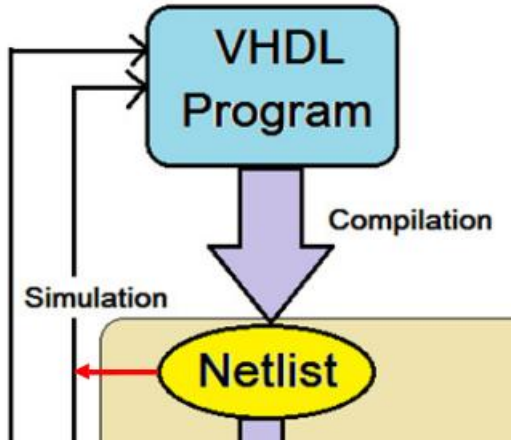


1. **Scrittura del codice VHDL**: si scrive il file .vhdl (o .vhd) del design.
2. **Scrittura del testbench**: si crea un file VHDL separato (il "testbench") che serve a:
   - istanziare (collegare) il design che vogliamo testare;
   - generare i segnali di ingresso (stimoli) per il design;
   - monitorare le uscite.
3. **Analisi**: il codice VHDL viene esaminato per capire se sono presenti errori e per produrre un file in formato intermedio (che può entrare a far parte di una libreria di componenti).
4. **Elaborazione**: il codice VHDL del design e del testbench viene compilato in un formato direttamente eseguibile dal simulatore.
5. **Esecuzione** del file prodotto dalla precedente elaborazione (**simulazione**): viene eseguito il testbench, si applicano gli stimoli e viene salvato l'andamento dei segnali nel tempo (forme d'onda).
6. **Analisi delle forme d'onda**: si verifica (usando un apposito **visualizzatore**) se le uscite corrispondono al comportamento atteso.

Diversi strumenti di simulazione disponibili:

- GHDL (gratuito e open source)
- ModelSim (o QuestaSim)
- Vivado Simulator (integrato nell'ambiente Xilinx)
- Quartus Prime Simulator (integrato nell'ambiente Intel/Altera)

Strumenti necessari



I punti della slide precedente li realizzeremo con **due strumenti** utilizzabili solo da riga di comando (dalla shell).

Essi sono:

- **ghdl**: http://ghdl.free.fr/ strumento multi funzione VHDL
- **gtkwave**: https://gtkwave.sourceforge.net/ la fase di analisi via testbench produce un file onda. Questo strumento è il suo visualizzatore. Dei due tool è l'unico che ha un'interfaccia grafica.

Primo esercizio VHDL

Obiettivo: descrivere in VHDL, **simulare** con ghdl e fare **l'analisi d'onda** in gtkwave di una porta AND.

1. **Scrittura del codice VHDL**: si scrive il file .vhdl (o .vhd) del design.
2. **Scrittura del testbench**: si crea un file VHDL separato (il "testbench")
3. **Analisi**: il codice VHDL viene esaminato per capire se sono presenti errori e per produrre un file in formato intermedio (che può entrare a far parte di una libreria di componenti).
4. **Elaborazione**: il codice VHDL del design e del testbench viene compilato in un formato direttamente eseguibile dal simulatore.
5. **Esecuzione** del file prodotto dalla precedente elaborazione (**simulazione**): viene eseguito il testbench, si applicano gli stimoli e viene salvato l'andamento dei segnali nel tempo (forme d'onda).
6. **Analisi delle forme d'onda**: si verifica (usando un apposito **visualizzatore**) se le uscite corrispondono al comportamento atteso.

Primo esercizio VHDL

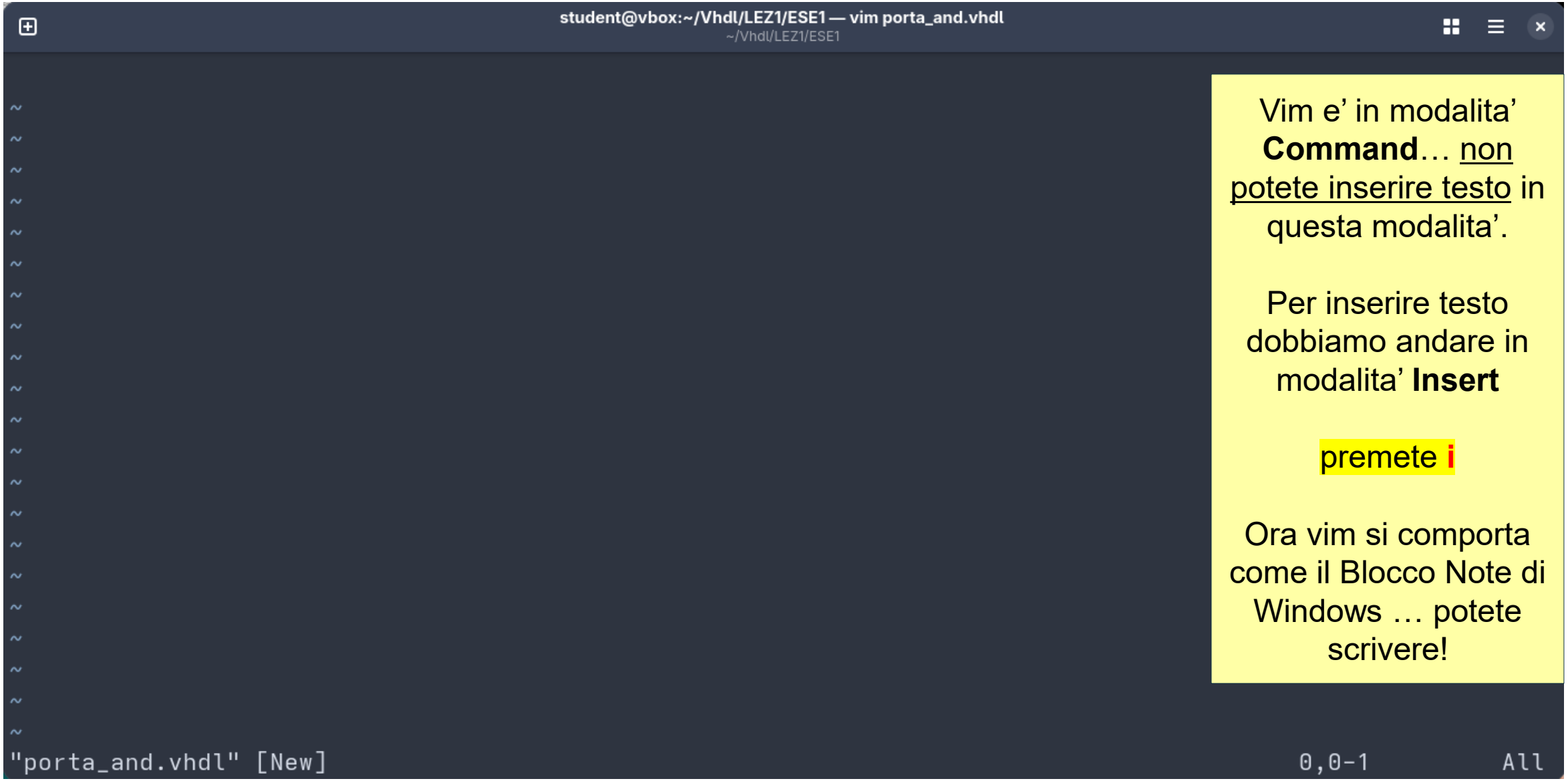
1. Scrittura del codice VHDL

Regola: mettere ogni **entity** in un file differente. Il file **deve** avere lo stesso nome della entity. Qui abbiamo una sola entity = un solo file.

- Recatevi nella cartella home — **cd**
- Muoversi nella sottocartella ./LEZ1/ESE1 — **cd ~/LEZ1/ESE1**
- Scrivete vim porta_and.vhdl e premete invio — **vim porta_and.vhdl**

Vi troverete in vim con la schermata visibile nella prossima slide …

Primo esercizio VHDL



Vim e' in modalita' **Command**… non potete inserire testo in questa modalita'.

Per inserire testo dobbiamo andare in modalita' **Insert**

premete **i**

Ora vim si comporta come il Blocco Note di Windows … potete scrivere!

Primo esercizio VHDL

student@vbox:~/Vhdl/LEZ1/ESE1 — vim porta_and.vhdl
~/Vhdl/LEZ1/ESE1

```vhdl
-- Dichiarazione delle librerie
library ieee;
use ieee.std_logic_1164.all;

-- Definizione dell'entity: interfaccia esterna della porta AND
entity porta_and is
    port (
        A : in  std_logic;  -- Ingresso 1
        B : in  std_logic;  -- Ingresso 2
        Y : out std_logic   -- Uscita
    );
end entity porta_and;

-- Definizione dell'architecture: implementazione del comportamento della porta AND
architecture arch_porta_and of porta_and is
begin
    -- Assegnazione concorrente: l'uscita Y è il risultato dell'AND logico tra A e B
    Y <= A and B;
end architecture arch_porta_and;
~
```

Scrivete nel file quello che e' visualizzato in questa slide...

Primo esercizio VHDL

student@vbox:~/Vhdl/LEZ1/ESE1 — vim porta_and.vhdl
~/Vhdl/LEZ1/ESE1

```
-- Dichiarazione delle librerie
library ieee;
use ieee.std_logic_1164.all;

-- Definizione dell'entity: interfaccia esterna della porta AND
entity porta_and is
    port (
        A : in  std_logic;  -- Ingresso 1
        B : in  std_logic;  -- Ingresso 2
        Y : out std_logic   -- Uscita
    );
end entity porta_and;

-- Definizione dell'architecture: implementazione del comportamento della porta AN
architecture arch_porta_and of porta_and is
begin
    -- Assegnazione concorrente: l'uscita Y è il risultato dell'AND logico tra A e
    Y <= A and B;
end architecture arch_porta_and;
~
~
~
```

Vim CommandLine

```
:wq
```

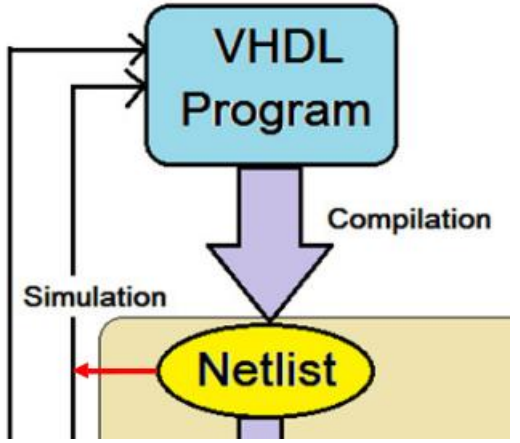
Ok, il file e' scritto … e ora **COME LO SALVO**???

Per salvare il file dobbiamo cambiare modalita'. Dobbiamo andare in **CommandLine mode** ma non possiamo saltare direttamente da Insert mode. Dobbiamo passare per Command mode.

Scrivete:
Esc:wq

E premete **invio**. Vim salva il file e si chiude.

Primo esercizio VHDL

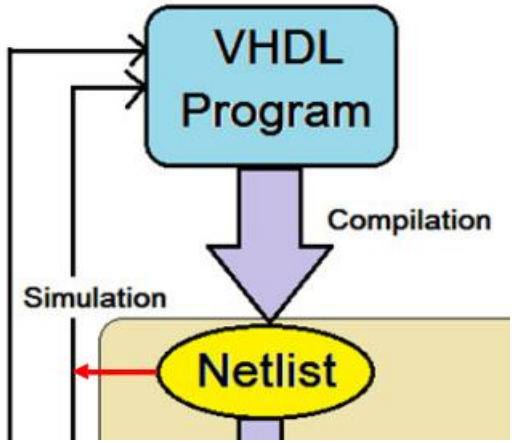


2. Scrittura del testbench

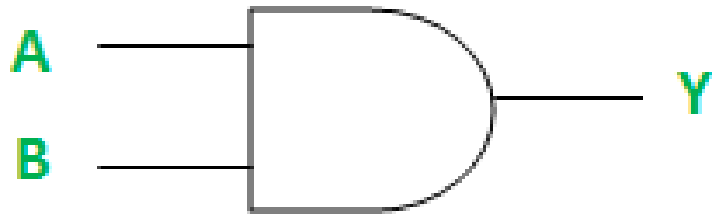
La stesura di un testbench da zero richiede ulteriore conoscenza. Forniamo noi un file testbench di prova che configureremo per testare la porta AND

Il file è presente in ./LEZ1/ESE1 con il nome di testbench_porta_and.vhdl

Primo esercizio VHDL



Prima di inserire il codice del test (che non è altro che una impostazione **SEQUENZIALE** dei valori in ingresso di porta_and) rivediamo brevemente la porta AND e la sua tabella di verità :



A	B	Y = A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Questi sono i valori di segnale che dovremo impostare. Dopo OGNI impostazione dei segnali in input dovremo dire a VHDL di aspettare per una certa quantità di tempo. Facciamo, come esempio, **5 nanosecondi**.

Primo esercizio VHDL

student@vbox:~/Vhdl/LEZ1/ESE1 — vim testbench_porta_and.vhdl
~/Vhdl/LEZ1/ESE1

```
    -- Map le porte del componente alle variabili/segnali del testbench
    DUT : porta_and
    port map (
        A => A,
        B => B,
        Y => Y
    );

    -- Processo di generazione degli stimoli
    stimulus_process : process
    begin
        -- Test Cases here

        wait; -- Mantiene il processo inattivo indefinitamente dopo l'ultimo stimolo

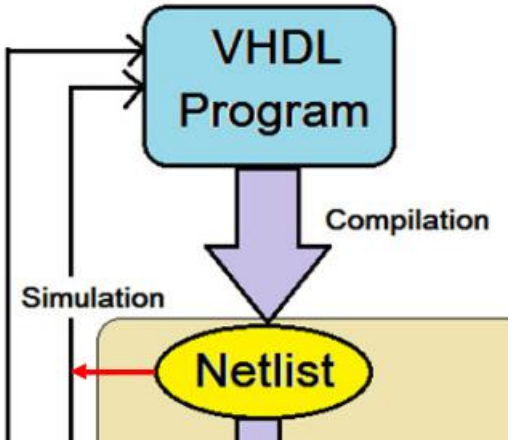
    end process stimulus_process;

end architecture test;
```

Inserite il seguente codice VHDL dove indicato (poi salvate e uscite da vim)

```
a<='0';b<='0';
wait for 5 ns;
a<='0';b<='1';
wait for 5 ns;
a<='1';b<='0';
wait for 5 ns;
a<='1';b<='1';
wait for 5 ns;
```

Primo esercizio VHDL



3. Analisi Come primo step dobbiamo effettuare l'**analisi** del file della struttura e comportamento della porta and.

Questo step si realizza utilizzando ghdl come segue:

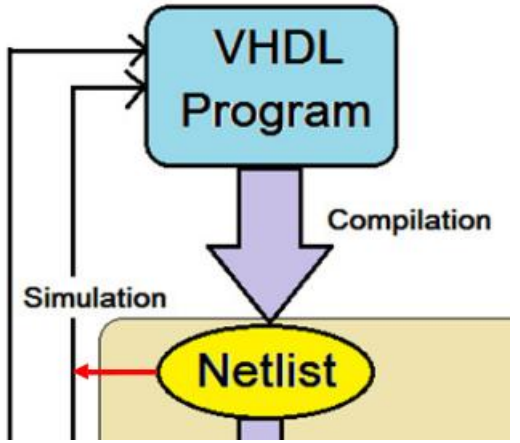
```
student@vbox:~/Vhdl/LEZ1/ESE1
~/Vhdl/LEZ1/ESE1
student@vbox:~/Vhdl/LEZ1/ESE1$ ls
porta_and.vhdl  testbench_porta_and.vhdl
student@vbox:~/Vhdl/LEZ1/ESE1$ ghdl -a porta_and.vhdl
student@vbox:~/Vhdl/LEZ1/ESE1$ ls
porta_and.o  porta_and.vhdl  testbench_porta_and.vhdl  work-obj93.cf
student@vbox:~/Vhdl/LEZ1/ESE1$
```

Il parametro **-a** di ghdl vuol dire: **analizza** . Produce un file **.o** (formato oggetto) che viene collegato ad una libreria di nome **work** posizionata nella cartella corrente (current folder) **.cf** .

Se dovessimo scrivere altri file vhdl nella stessa cartella e volessimo utilizzare il componente porta_and ora potremmo farlo (data la sua presenza in **work**).

Primo esercizio VHDL

Ripetiamo l'analisi anche per il testbench




```
student@vbox:~/Vhdl/LEZ1/ESE1
~/Vhdl/LEZ1/ESE1
student@vbox:~/Vhdl/LEZ1/ESE1$ ls
porta_and.o  porta_and.vhdl  testbench_porta_and.vhdl  work-obj93.cf
student@vbox:~/Vhdl/LEZ1/ESE1$ ghdl -a testbench_porta_and.vhdl
student@vbox:~/Vhdl/LEZ1/ESE1$ ls
porta_and.o  porta_and.vhdl  testbench_porta_and.o  testbench_porta_and.vhdl  work-obj93.cf
student@vbox:~/Vhdl/LEZ1/ESE1$
```

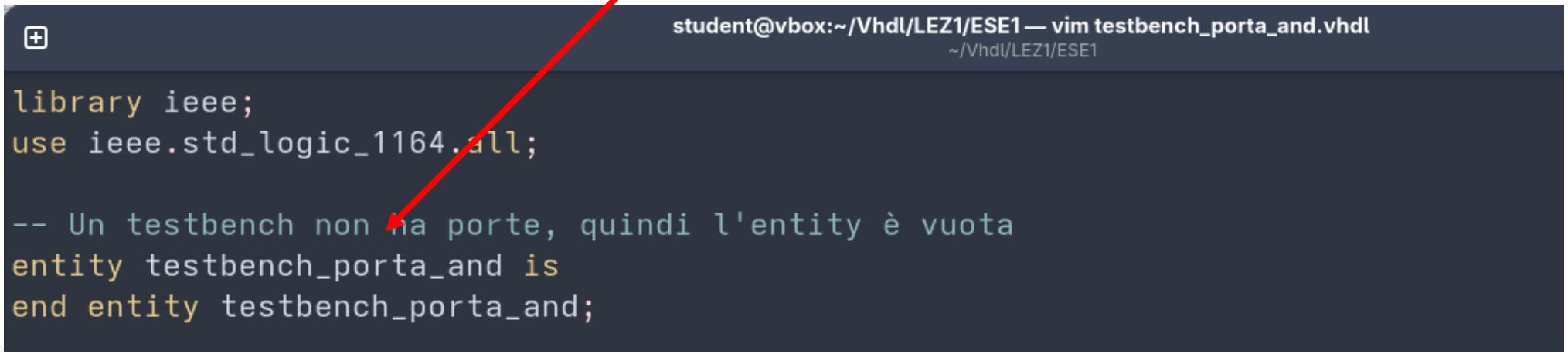
Primo esercizio VHDL

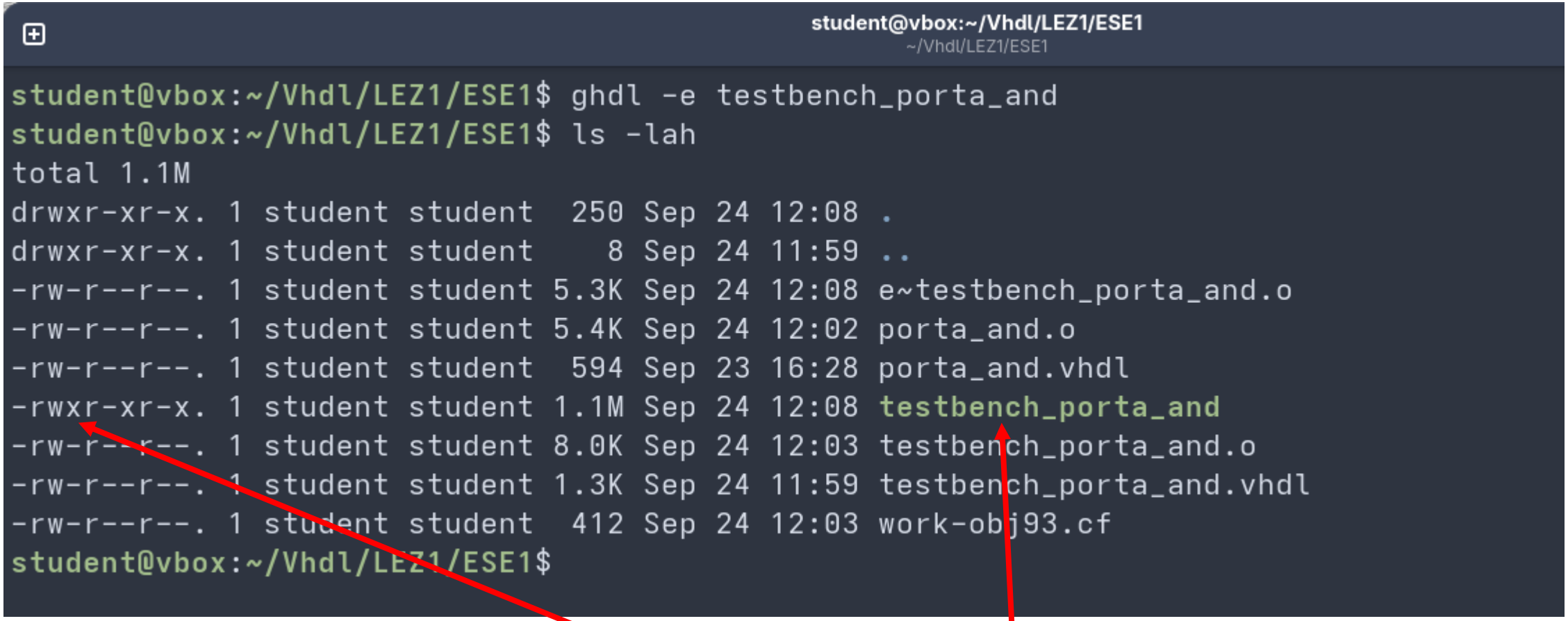
4. Elaborazione

Il design da «eseguire» è il testbench. Andiamo a dire a ghdl di crearci un esegubile per il testbench:

- ghdl **-e** testbench_porta_and → **elaborazione**: produce un eseguibile

Nome del testbench, non il nome del file





File eseguible

Primo esercizio VHDL

5. Esecuzione Abbiamo prodotto l'eseguibile del testbench. Ci accorgiamo che e' un file eseguibile poiche' i suoi permessi sono i seguenti (si vede da output **ls -lah**):

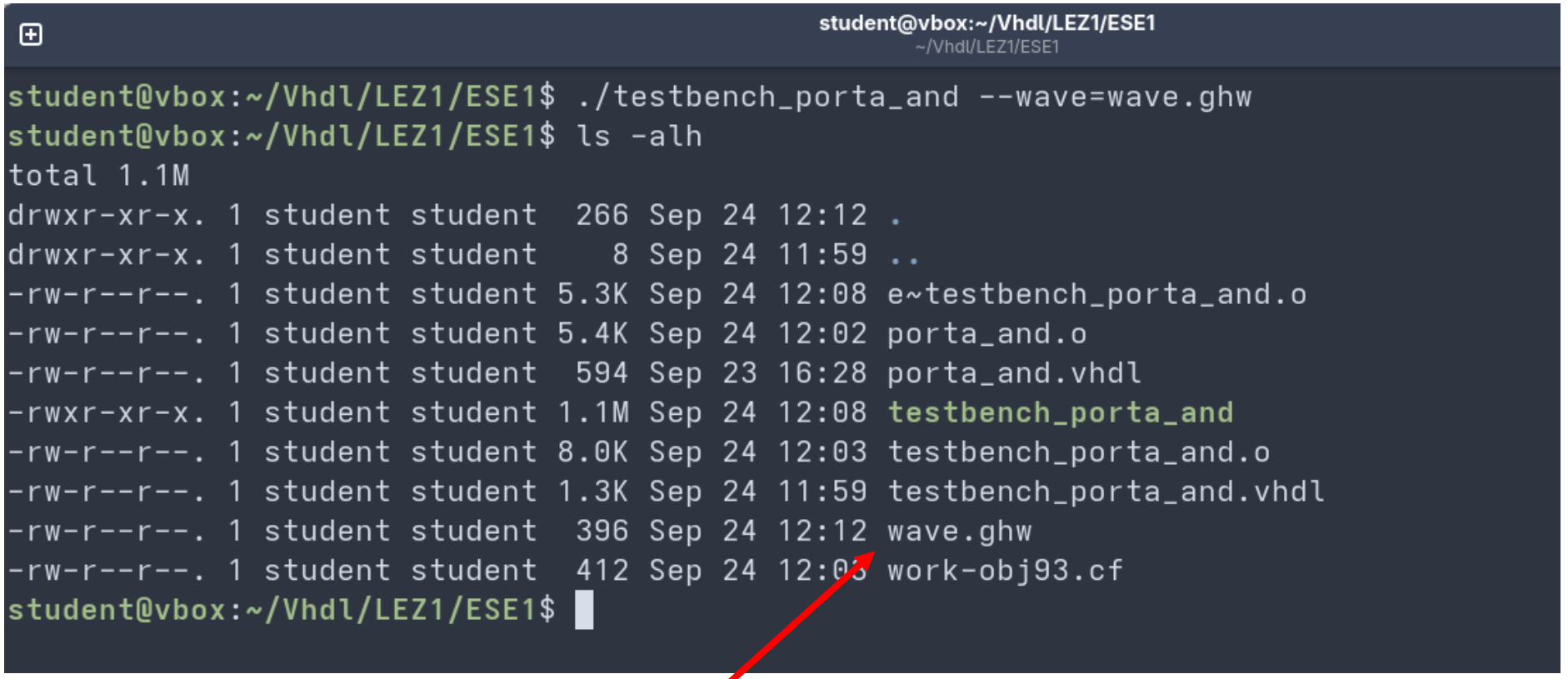
-rw**x**r-**x**r-**x**. 1 student student 1.3M Aug 9 17:48 **testbench_porta_and**

- Il primo carattere NON E' d (directory) ma e' **–** (file regolare)
- TUTTE LE CATEGORIE DI PERMESSI (utente/gruppo dell'utente/resto del mondo) hanno una **x** (e**x**ecute) … ossia hanno il permesso di eseguire il file

Ma … fermi tutti … **COME si fa ad ESEGUIRE il file**? Si scrive **./nomefiledaeseguire** e si preme invio.

Dato che stiamo lavorando con VHDL eseguire questo file ha un significato preciso … eseguire il file testbench equivale a **SIMULARE** la entity!

Primo esercizio VHDL



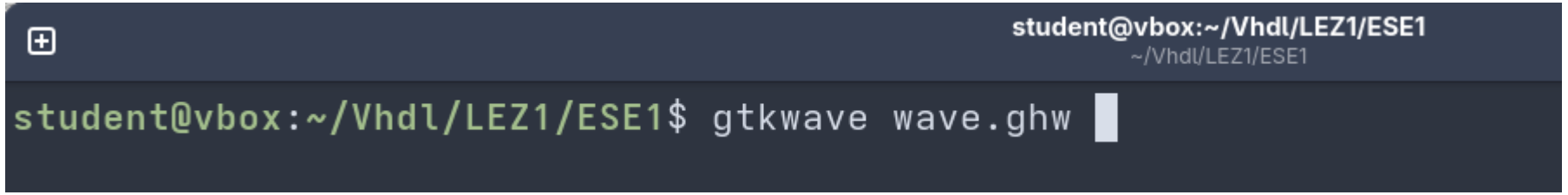
Il file eseguibile prodotto (quello che permette di simulare) ha dei parametri. Tra questi, a noi interessa quello che permette di salvare in un file l'analisi d'onda della entità simulata. La simulazione si effettua con il seguente comando:

./testbench_porta_and --wave=wave.ghw → produce il file ghw da ispezionare

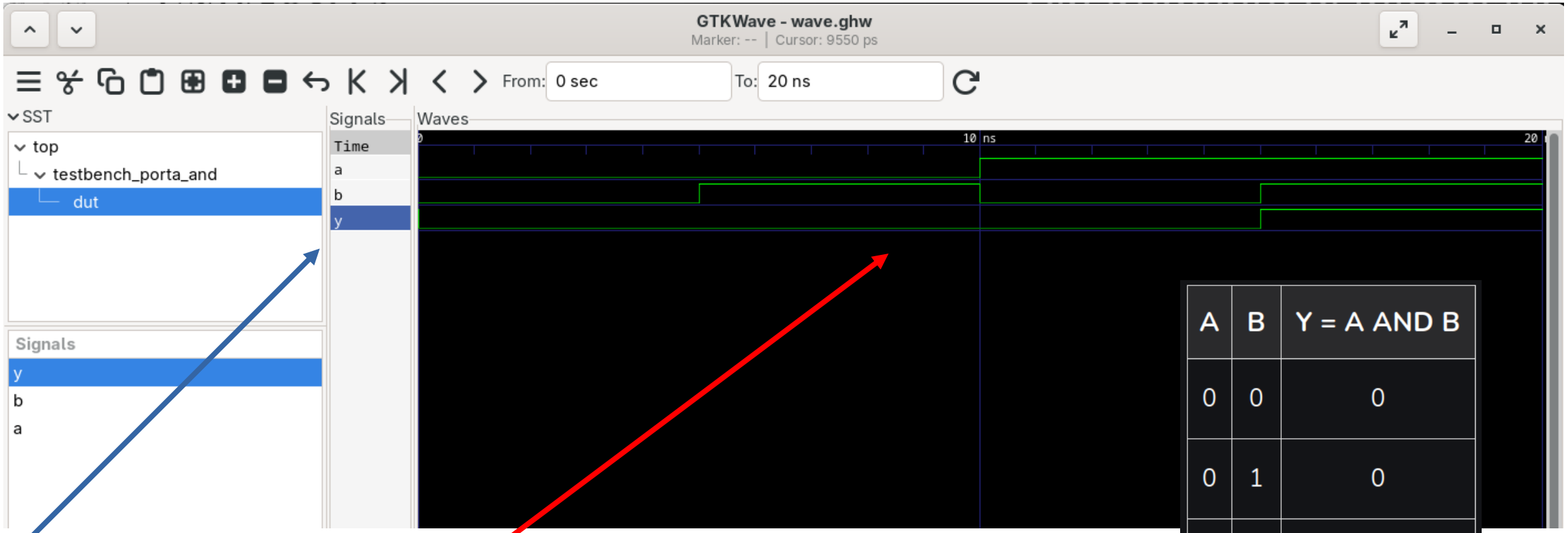
Primo esercizio VHDL

6. Analisi delle forme d'onda Ultimo step dell'esercizio :) … Dobbiamo ispezionare il file ghw delle forme d'onda (andamento dei segnali che attraversano la entity di interesse) durante l'esecuzione del testbench E CONFRONTARE I CORRISPONDENTI SEGNALI IN USCITA CON LA TABELLA DI VERITA' DELLA ENTITY.

Per fare questo utilizzeremo **gtkwave** come segue:



Primo esercizio VHDL



A	B	Y = A AND B
0	0	0
0	1	0
1	0	0
1	1	1

Qui trovate i segnali testati

Confrontate il loro valore con il contenuto della TDV

Questo conclude l'esercizio

Concetto di concorrenza

- Le assegnazioni con operatori logici (and, or, not, xor, ecc.) sono **dichiarazioni concorrenti**.
- Esse descrivono la logica che è **sempre attiva**: qualsiasi cambiamento in un segnale di ingresso a destra dell'operatore (*Data_A* o *Data_B*) provoca un ricalcolo del risultato (*Result_Y*).

Result_Y <= *Data_A* and *Data_B*;

Caratteristica	**Programmazione software (sequenziale)**	**VHDL (concorrente/parallelo)**
Natura	Esecuzione sulla CPU	Descrizione di un circuito fisico
Flusso	Le istruzioni sono eseguite una dopo l'altra (linea 1, poi linea 2, poi linea 3...).	Tutte le dichiarazioni concorrenti sono sempre attive ed eseguite simultaneamente.
Dipendenza	L'ordine delle istruzioni è cruciale.	L'ordine delle dichiarazioni concorrenti è irrilevante per la logica (ma cruciale per la leggibilità).
Tempo	Il tempo è misurato in cicli di clock o millisecondi di esecuzione.	Il tempo è misurato in ritardi di propagazione (nanosecondi).

Assegnazione concorrente

- Modo per assegnare valori ai segnali in modo che l'operazione avvenga simultaneamente, non in sequenza.

- Si usa principalmente per descrivere la logica combinatoria, cioè circuiti in cui l'uscita dipende solo dagli ingressi attuali.

- Sintassi: *segnale_di_uscita* <= *espressione*;
  - *segnale_di_uscita*: il segnale a cui si desidera assegnare un valore.
  - <=: l'operatore di assegnazione del segnale.
  - *espressione*: può essere un valore, un altro segnale o una combinazione logica di segnali (es. A and B).

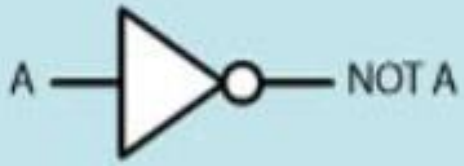
- **Esecuzione in parallelo**: ogni volta che uno dei segnali presenti nell'espressione a destra cambia valore, l'intera espressione viene rivalutata e il risultato viene assegnato al segnale di uscita.
  - Comportamento che rispecchia la natura parallela e simultanea dell'hardware.

Porte logiche fondamentali

Per ogni porta:

- **tabella di verità:** tutti i possibili ingressi e le relative uscite.
- **espressione booleana:** la formula matematica.
- **codice VHDL semplice:** come si descrive quella porta in VHDL (usando l'assegnazione concorrente, come fatto per la AND).

Porta logica NOT



INPUT	OUTPUT
A	NOT A
0	1
1	0

- Espressione: Y=!A
- Y <= not A;

Porta logica AND



INPUT		OUTPUT
A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

- Espressione: Y=AB
- Y <= A and B;

Porta logica OR



INPUT		OUTPUT
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

- Espressione: Y=A+B
- Y <= A or B;

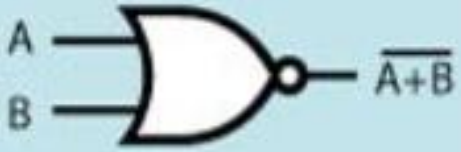
Porta logica NAND



INPUT		OUTPUT
A	B	$\overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

- Espressione: $Y=\overline{AB}$
- Y <= A nand B;

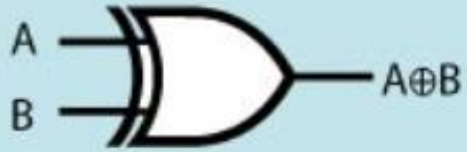
Porta logica NOR



INPUT		OUTPUT
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

- Espressione: $Y= \overline{A+B}$
- Y <= A nor B;

Porta logica XOR



INPUT		OUTPUT
A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

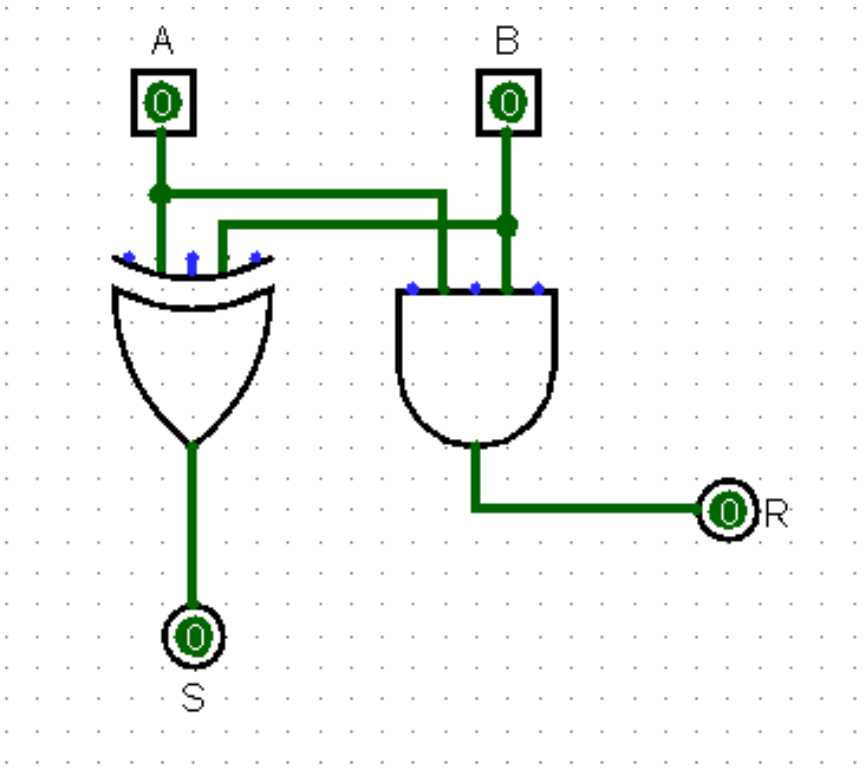
- Espressione: $Y=A \oplus B$
- Y <= A xor B;

Uso combinato di porte logiche

- Circuiti digitali composti da combinazioni di porte di base
- Esempio: circuito per la somma di un bit (Half Adder)
  - Ingressi: A, B
  - Uscite: S (Somma), R (Riporto)
  - Logica:
    - S <= A xor B;
    - R <= A and B;

Tabella di verità half adder a 1 bit

A	B	S	R
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



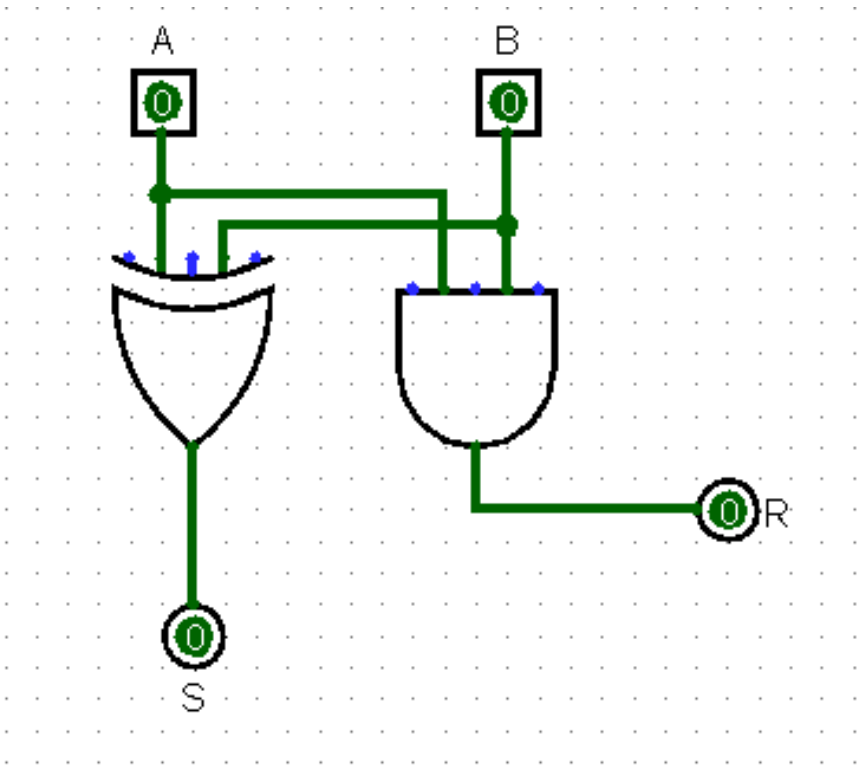
Secondo esercizio

- **Obiettivo**: descrivere in VHDL e simulare il comportamento di un half adder a 1 bit

- Ingressi: A, B
- Uscite: S (Somma), R (Riporto)
- Logica:
  - S <= A xor B;
  - R <= A and B;

Tabella di verità half adder a 1 bit

A	B	S	R
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



Secondo esercizio

- Muoversi nella cartella LEZ1/ESE2 e creare il file half_adder.vhdl
- Riempire i casi di test nel testbench fornito (file tb_ha.vhdl)
- Analisi entrambi i file e correggere eventuali errori
- Elaborare il testbench
- Eseguire il testbench
- Verifica dei risultati

Secondo esercizio

student@vbox:~/Vhdl/LEZ1/ESE2 — vim half_adder.vhdl
~/Vhdl/LEZ1/ESE2

```
library ieee;
use ieee.std_logic_1164.all;

entity half_adder is
        port(   A, B: in std_logic;
                S, R: out std_logic);
end entity half_adder;

architecture arch of half_adder is
begin
        S <= A xor B;
        R <= A and B;
end architecture arch;
~
```

Secondo esercizio

```
-- Processo di generazione degli stimoli
stimulus_process : process
begin
    -- Test Cases here
    A <= '0'; B <= '0'; wait for 5 ns;
    A <= '0'; B <= '1'; wait for 5 ns;
    A <= '1'; B <= '0'; wait for 5 ns;
    A <= '1'; B <= '1'; wait for 5 ns;

    wait; -- Mantiene il processo inattivo indefinitamente dopo l'ultimo stimolo

end process stimulus_process;
```

Secondo esercizio

